



1. SISD、SIMD、MIMD的基本概念

1.1 Flynn分类



SISD

单指令流单数据流

SIMD

单指令流多数据流

并行处理

MISD*

多指令流单数据流

MIMD

多指令流多数据流

Single/Multiple Instruction stream
Single/Multiple Data stream

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD
单指令流单数据流

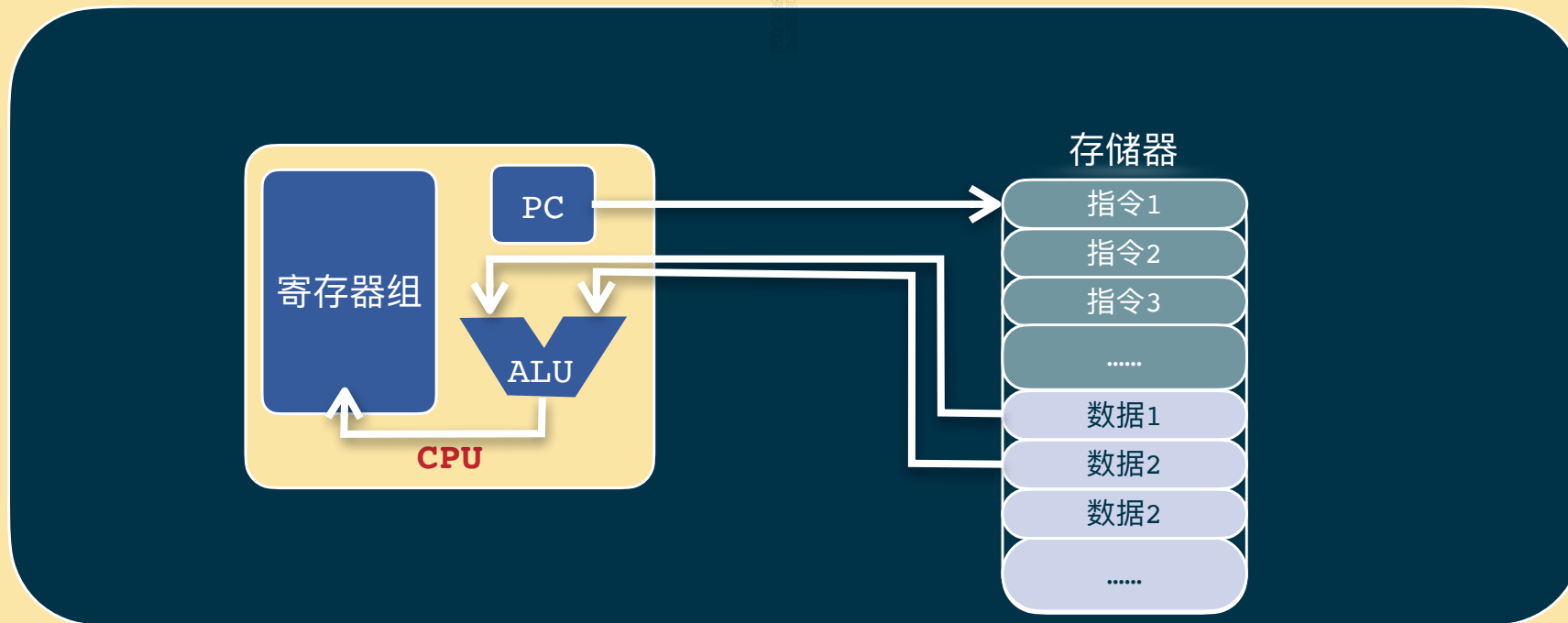
SIMD
单指令流多数据流

MISD*
多指令流单数据流

MIMD
多指令流多数据流



SISD是传统串行计算机，仅含一个处理器和一个存储器
顺序执行单条指令流，每条指令最多处理两个（双目）或一个（单目）数据
为提速，部分**SISD**采用指令流水线，可设多个功能部件和多模块交叉存储器



1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD
单指令流单数据流

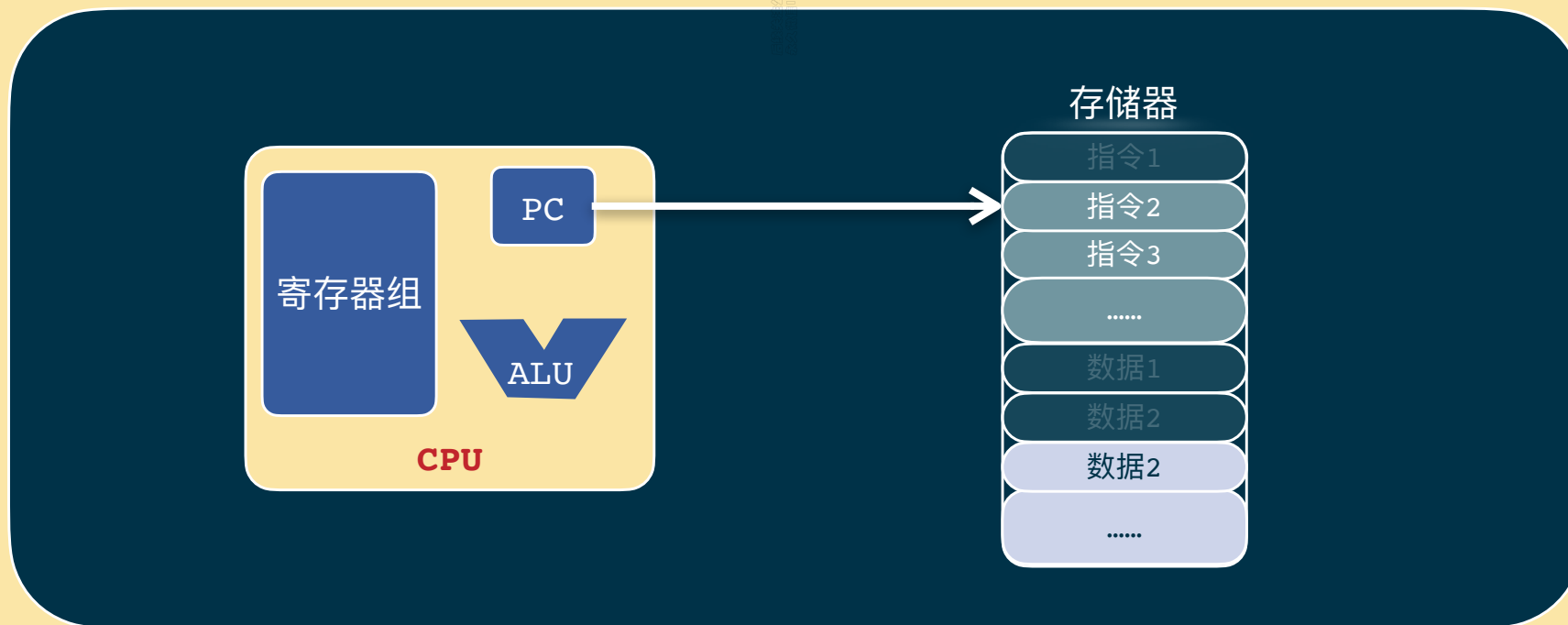
SIMD
单指令流多数据流

MISD*
多指令流单数据流

MIMD
多指令流多数据流



SISD是传统串行计算机，仅含一个处理器和一个存储器
顺序执行单条指令流，每条指令最多处理两个（双目）或一个（单目）数据
为提速，部分**SISD**采用指令流水线，可设多个功能部件和多模块交叉存储器



1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD
单指令流单数据流

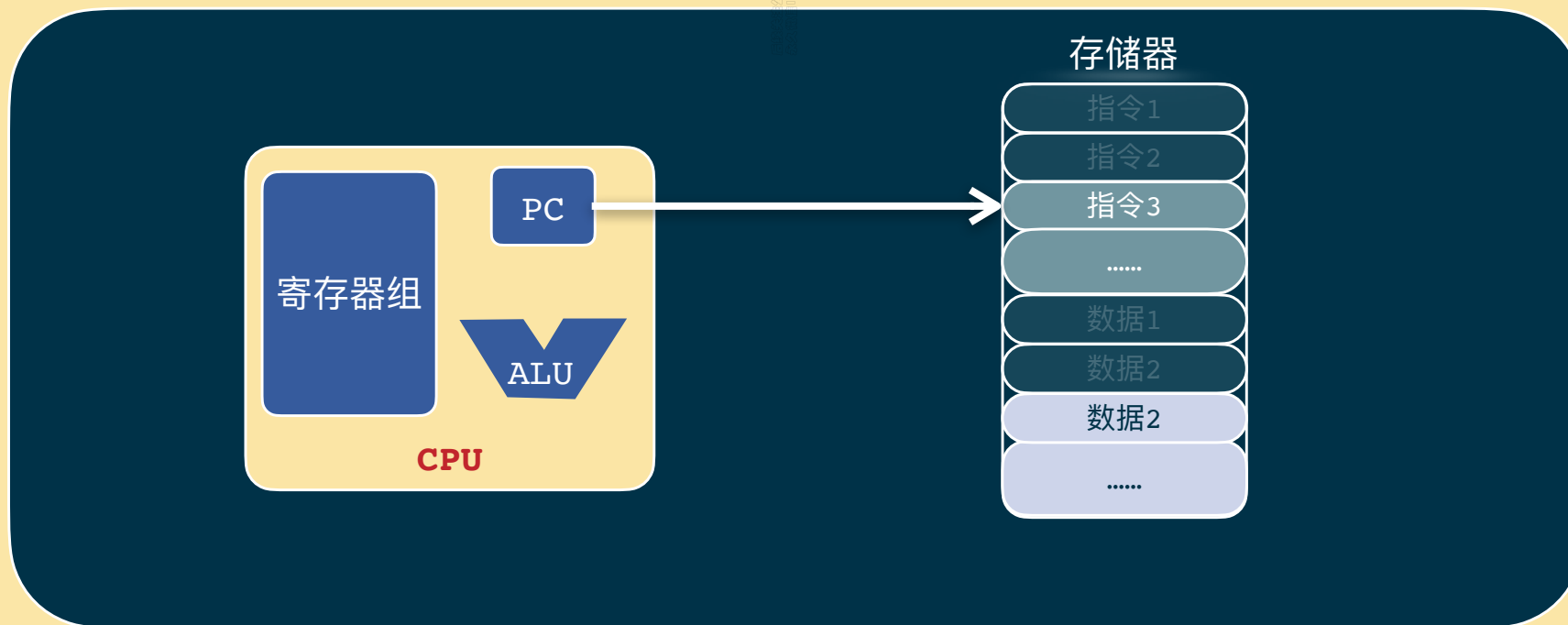
SIMD
单指令流多数据流

MISD*
多指令流单数据流

MIMD
多指令流多数据流



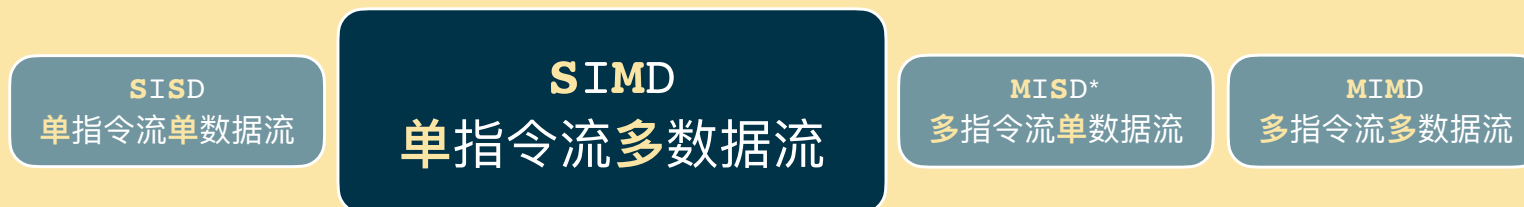
SISD是传统串行计算机，仅含一个处理器和一个存储器
顺序执行单条指令流，每条指令最多处理两个（双目）或一个（单目）数据
为提速，部分**SISD**采用指令流水线，可设多个功能部件和多模块交叉存储器



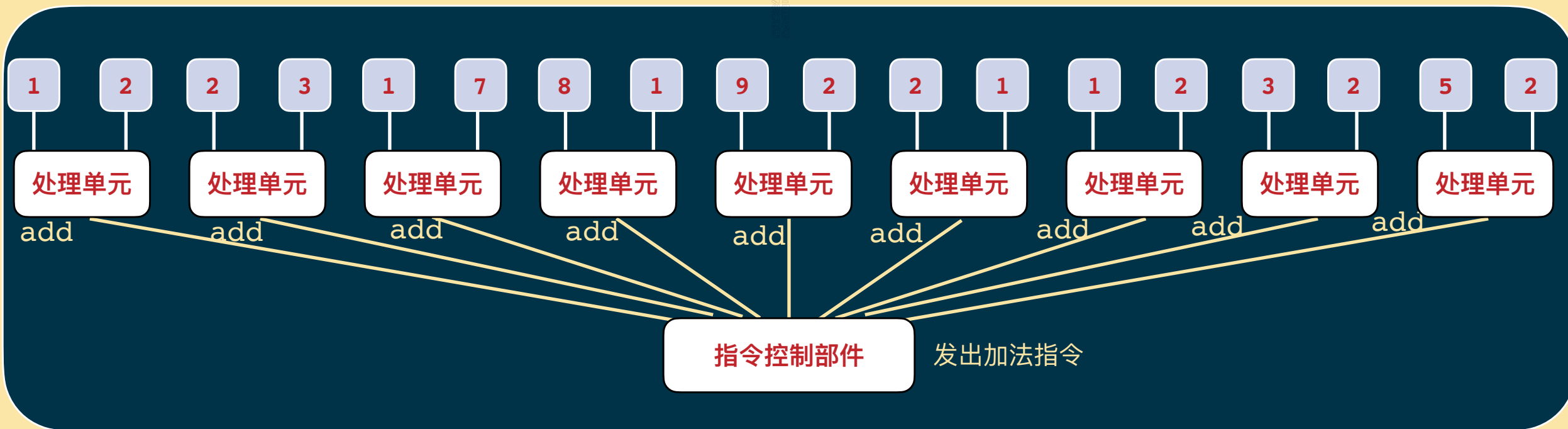
1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类



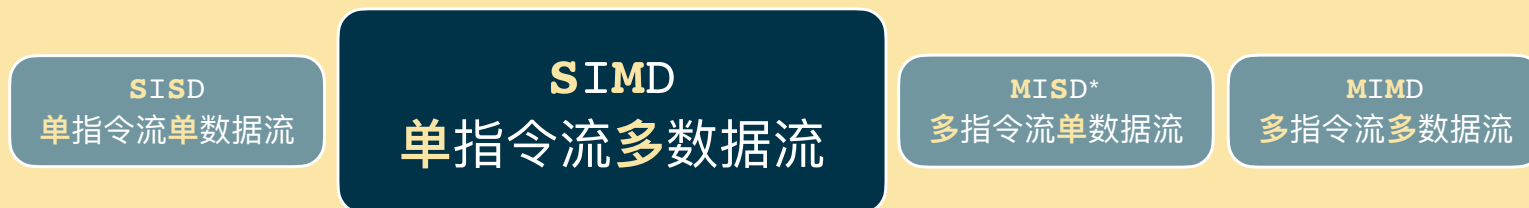
一条指令同时操作多个数据，非常适合于执行重复的相同操作，如图像处理中的像素操作
现代图形处理单元（GPU）在渲染3D图形时，会用SIMD处理成千上万个顶点和像素的相同操作



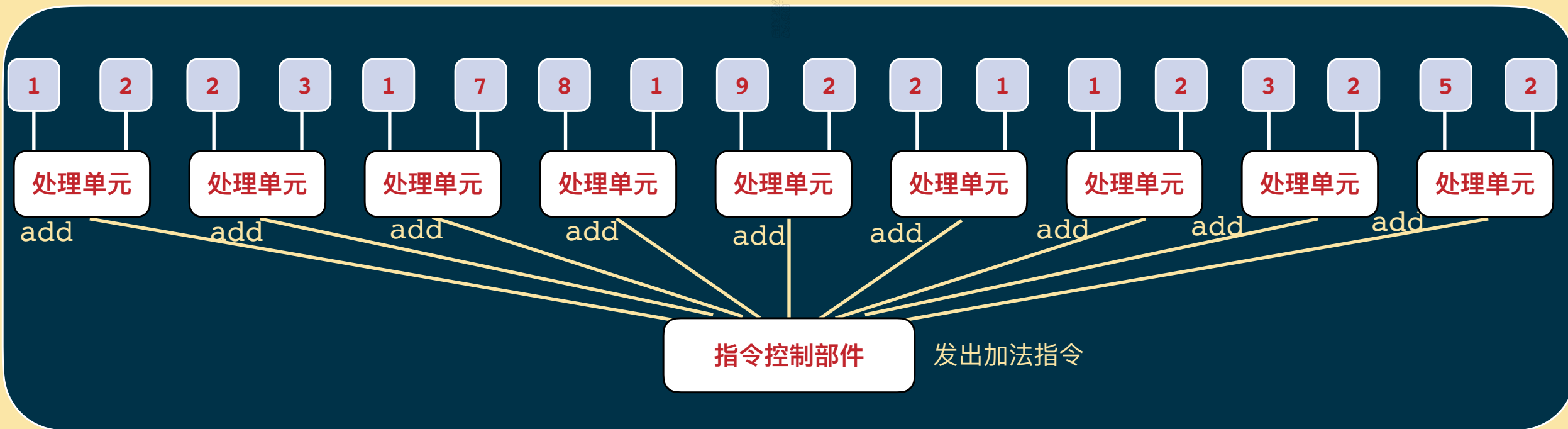
1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类



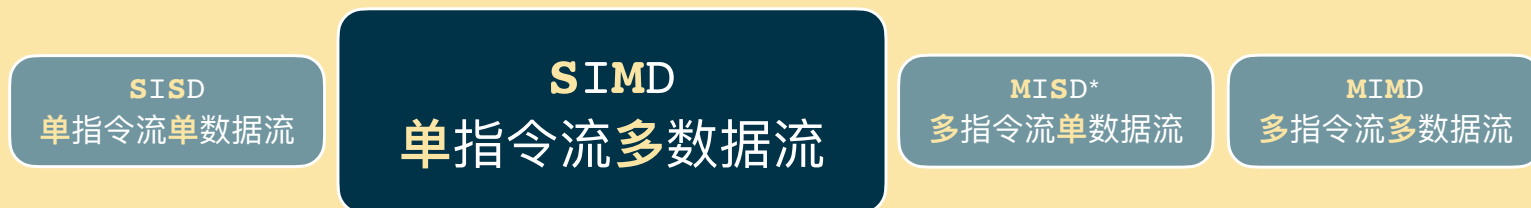
一条指令同时操作多个数据，非常适合于执行重复的相同操作，如图像处理中的像素操作
现代图形处理单元（GPU）在渲染3D图形时，会用SIMD处理成千上万个顶点和像素的相同操作



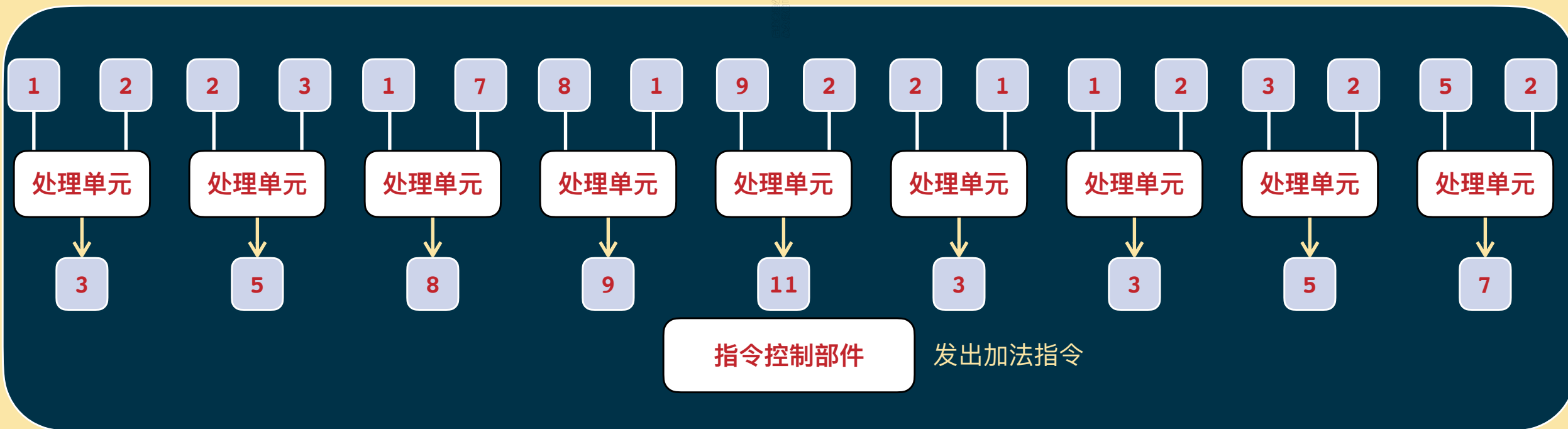
1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类



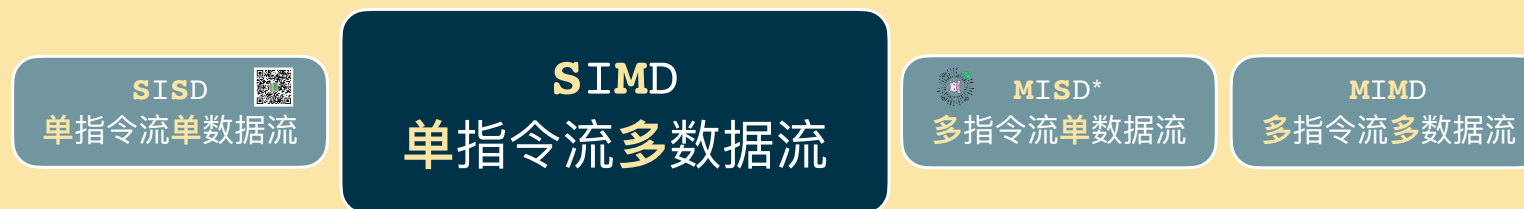
一条指令同时操作多个数据，非常适合于执行重复的相同操作，如图像处理中的像素操作
现代图形处理单元（GPU）在渲染3D图形时，会用SIMD处理成千上万个顶点和像素的相同操作





1. SISD、SIMD、MIMD的基本概念

1.1 Flynn分类



一条指令同时操作多个数据，非常适合于执行重复的相同操作，如图像处理中的像素操作
现代图形处理单元（GPU）在渲染3D图形时，会用SIMD处理成千上万个顶点和像素的相同操作

向量处理器

是一种实现了SIMD指令集的高性能计算机处理器，
它可以同时处理一组数据（向量）上的同一操作
这种处理器专为执行复杂数学运算而设计

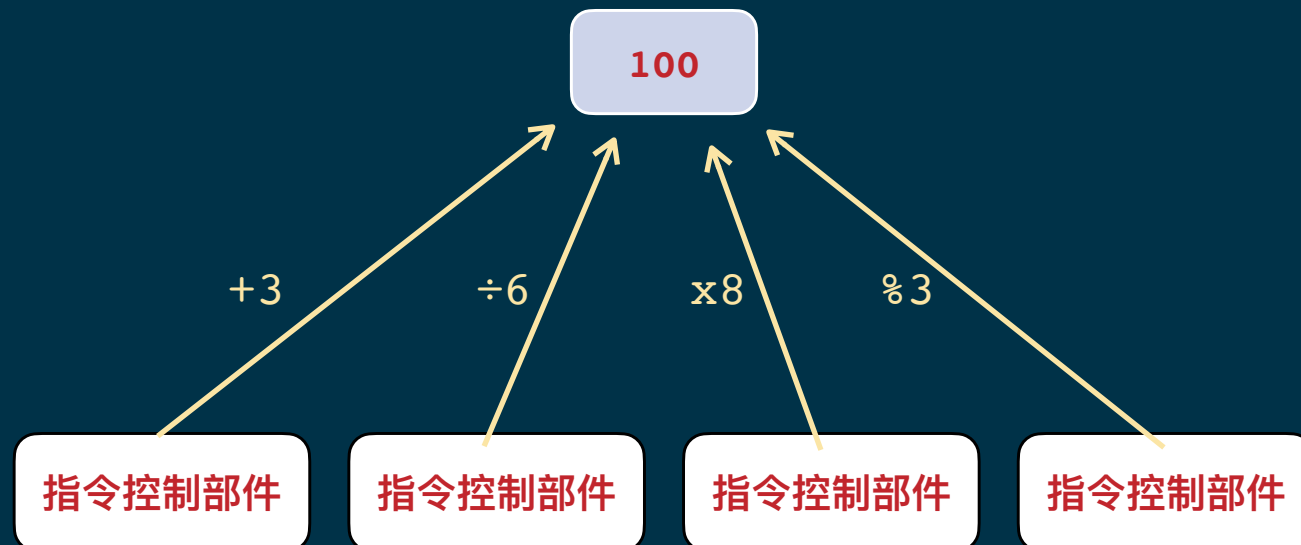
1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类



多条指令流同时操作同一数据
难以实现，实际上不存在



1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD

单指令流单数据流

SIMD

单指令流多数据流

MISD*

多指令流单数据流

MIMD

多指令流多数据流

多个处理器同时执行不同的指令序列，处理不同的数据

大型服务器在处理多用户的不同请求时会使用MIMD，例如web服务器或数据库服务器

数据1

数据2

数据3

处理器1

处理器2

处理器3

指令1

指令2

指令3

指令4

指令5

指令6

指令7

指令8

指令9

指令10

指令11

指令12

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD

单指令流单数据流

SIMD

单指令流多数据流

MISD*

多指令流单数据流

MIMD

多指令流多数据流

多个处理器同时执行不同的指令序列，处理不同的数据

大型服务器在处理多用户的不同请求时会使用MIMD，例如web服务器或数据库服务器

数据1

数据2

数据3

处理器1

处理器2

处理器3

指令1

指令5

指令9

指令2

指令6

指令10

指令3

指令7

指令11

指令4

指令8

指令12

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD

单指令流单数据流

SIMD

单指令流多数据流

MISD*

多指令流单数据流

MIMD

多指令流多数据流

多个处理器同时执行不同的指令序列，处理不同的数据

大型服务器在处理多用户的不同请求时会使用MIMD，例如web服务器或数据库服务器

数据1

数据2

数据3

处理器1

处理器2

处理器3

指令2

指令6

指令10

指令3

指令7

指令11

指令4

指令8

指令12

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD

单指令流单数据流

SIMD

单指令流多数据流

MISD*

多指令流单数据流

MIMD

多指令流多数据流

多个处理器同时执行不同的指令序列，处理不同的数据
大型服务器在处理多用户的不同请求时会使用MIMD，例如web服务器或数据库服务器

数据1

数据2

数据3

处理器1

处理器2

处理器3

指令3

指令7

指令11

指令4

指令8

指令12

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD

单指令流单数据流

SIMD

单指令流多数据流

MISD*

多指令流单数据流

MIMD

多指令流多数据流

多个处理器同时执行不同的指令序列，处理不同的数据

大型服务器在处理多用户的不同请求时会使用MIMD，例如web服务器或数据库服务器

数据1

数据2

数据3

处理器1

处理器2

处理器3

指令4

指令8

指令12

1. SISD、SIMD、MIMD的基本概念



1.1 Flynn分类

SISD
单指令流
单数据流

单打独斗
一个人干一件事

SIMD
单指令流
多数据流

听口令群体行动
发出指令，集体做一件事

MISD*
多指令流
单数据流

不能评价不存在的东西

MIMD
多指令流
多数据流

各忙各的
多人并发，互不干扰

Single/Multiple Instruction stream
Single/Multiple Data stream



2. 硬件多线程

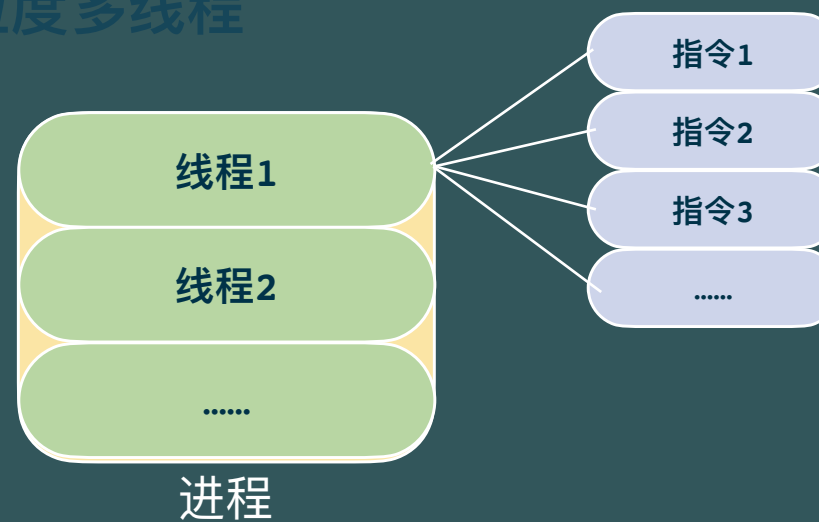
2.1 细粒度多线程



2. 硬件多线程

进程由很多线程组成，进程是资源分配的最小单位
线程是cpu调度的最小单位
操作系统第二章将会详细学习进程和线程的概念

2.1 细粒度多线程

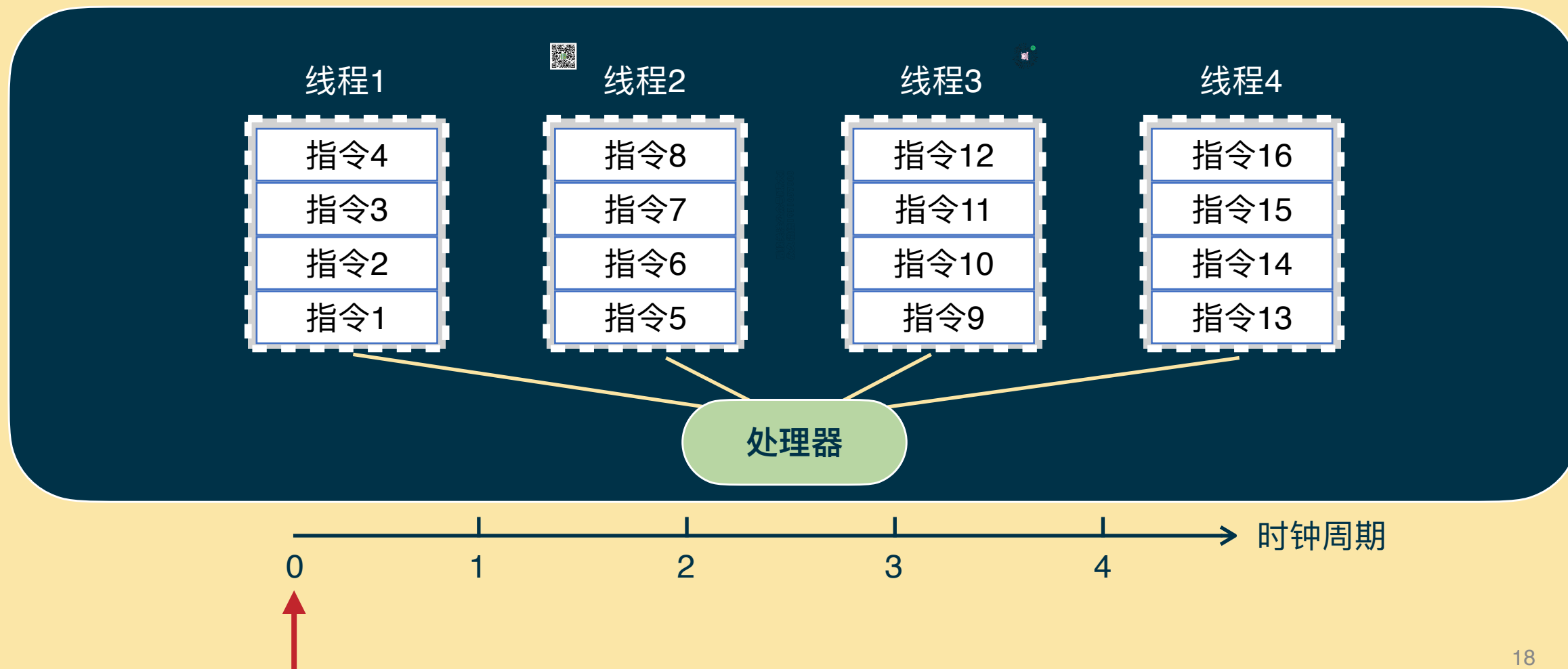




2. 硬件多线程

2.1 细粒度多线程

细粒度多线程允许处理器在每个时钟周期中轮换不同的线程
这种切换通常发生在一个线程执行指令时遇到长时间等待的情况下

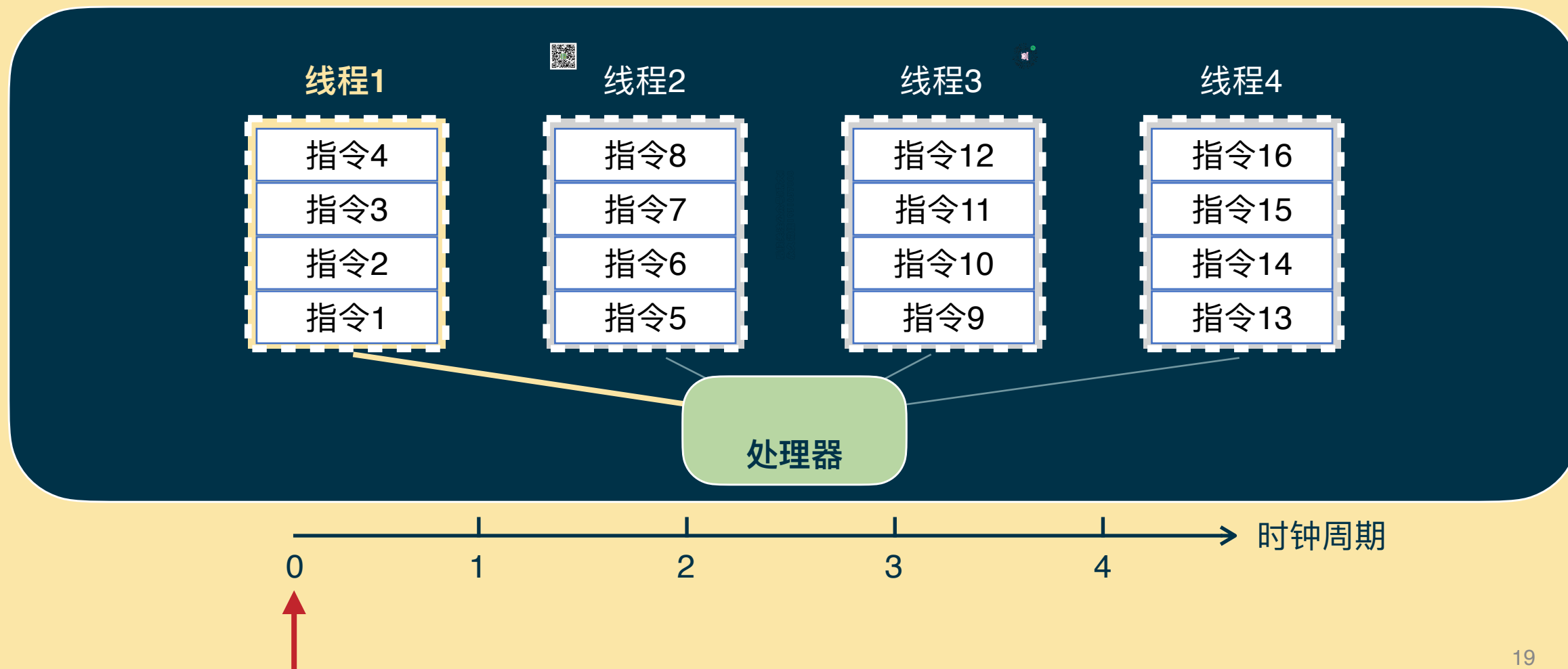




2. 硬件多线程

2.1 细粒度多线程

细粒度多线程允许处理器在每个时钟周期中轮换不同的线程
这种切换通常发生在一个线程执行指令时遇到长时间等待的情况下

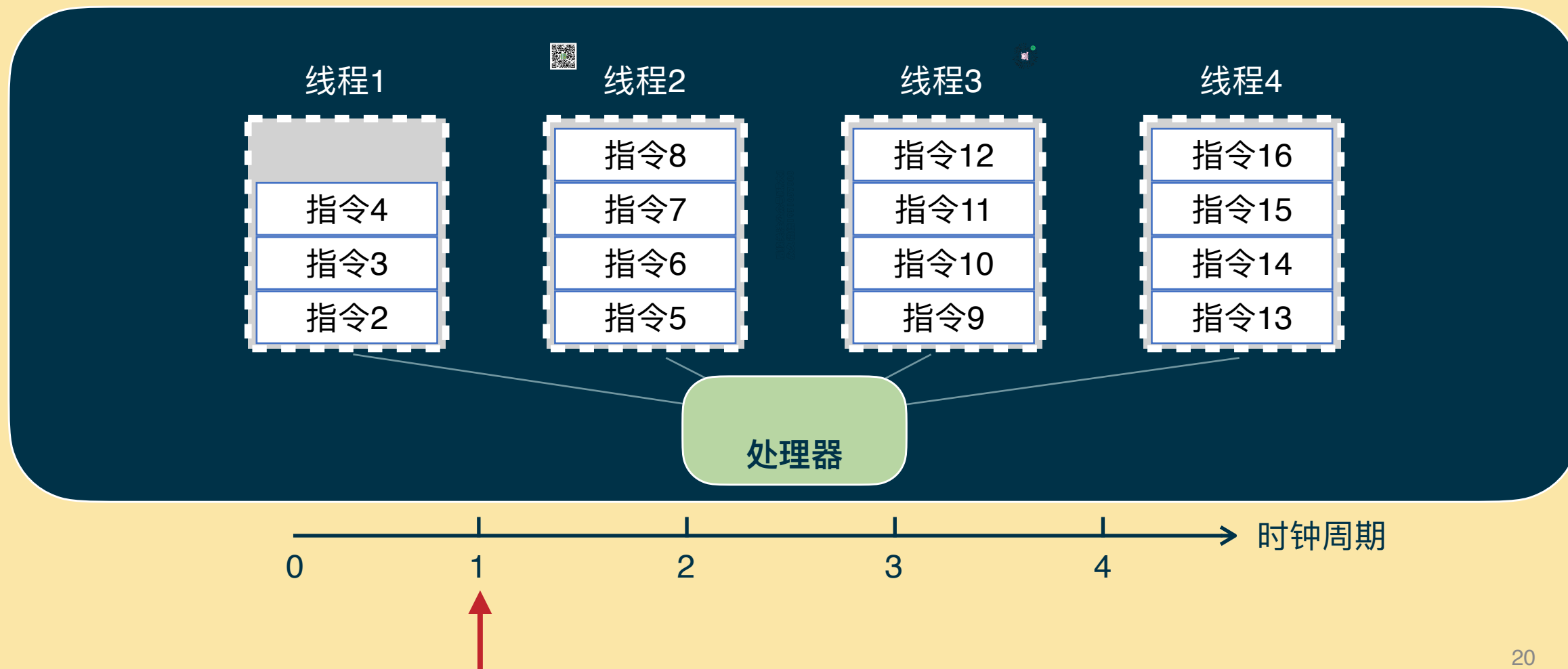




2. 硬件多线程

2.1 细粒度多线程

细粒度多线程允许处理器在每个时钟周期中轮换不同的线程
这种切换通常发生在一个线程执行指令时遇到长时间等待的情况下

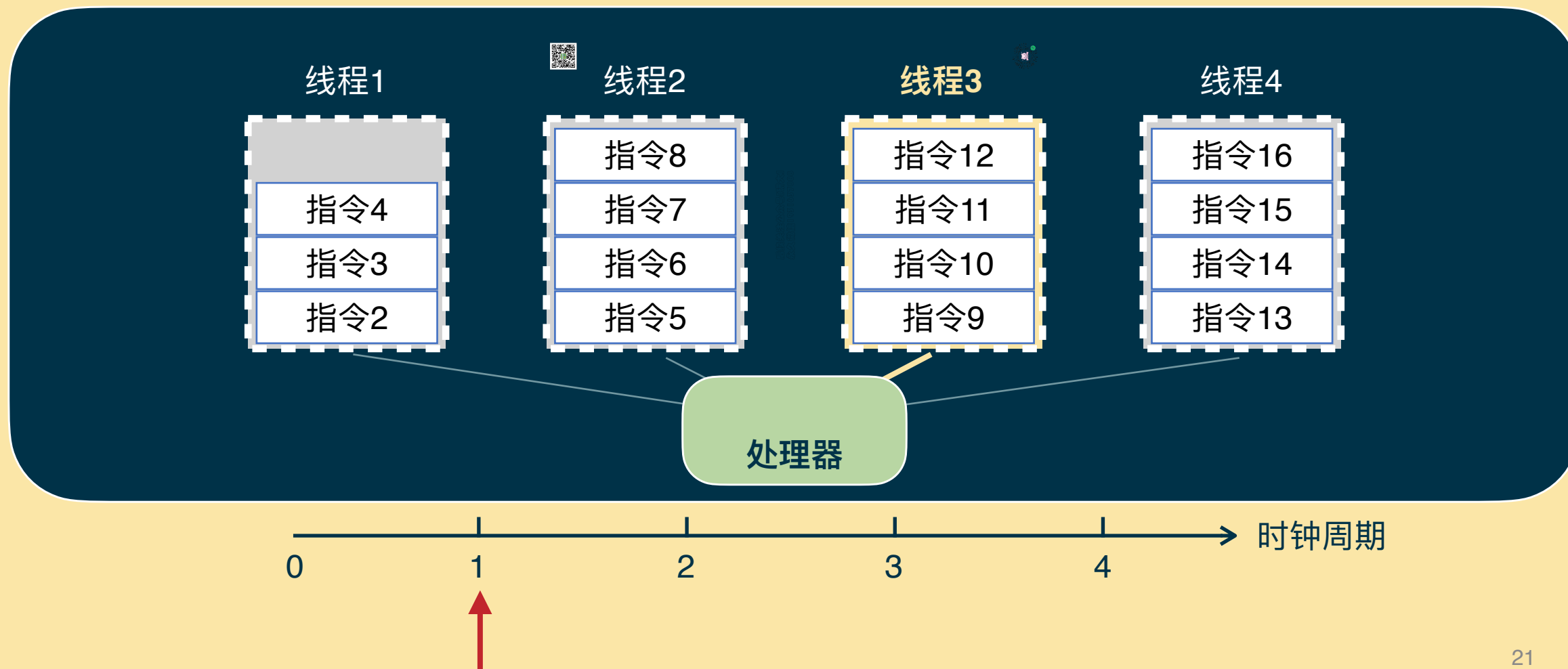




2. 硬件多线程

2.1 细粒度多线程

细粒度多线程允许处理器在每个时钟周期中轮换不同的线程
这种切换通常发生在一个线程执行指令时遇到长时间等待的情况下

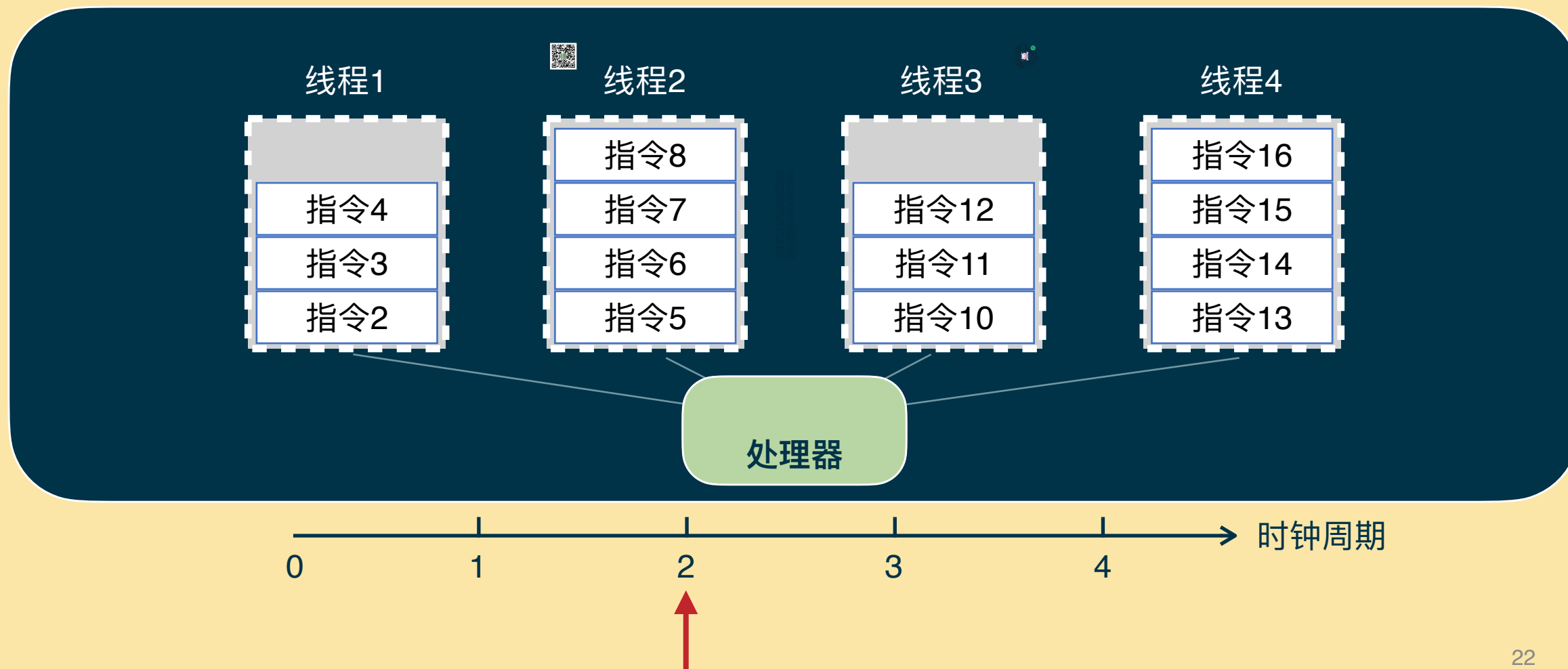




2. 硬件多线程

2.1 细粒度多线程

细粒度多线程允许处理器在每个时钟周期中轮换不同的线程
这种切换通常发生在一个线程执行指令时遇到长时间等待的情况下





2. 硬件多线程

2.2 粗粒度多线程

在当前执行的线程被阻塞或完成之后才切换到另一个线程
切换的原因通常是等待I/O操作、长时间的计算或其他资源的延迟



2. 硬件多线程

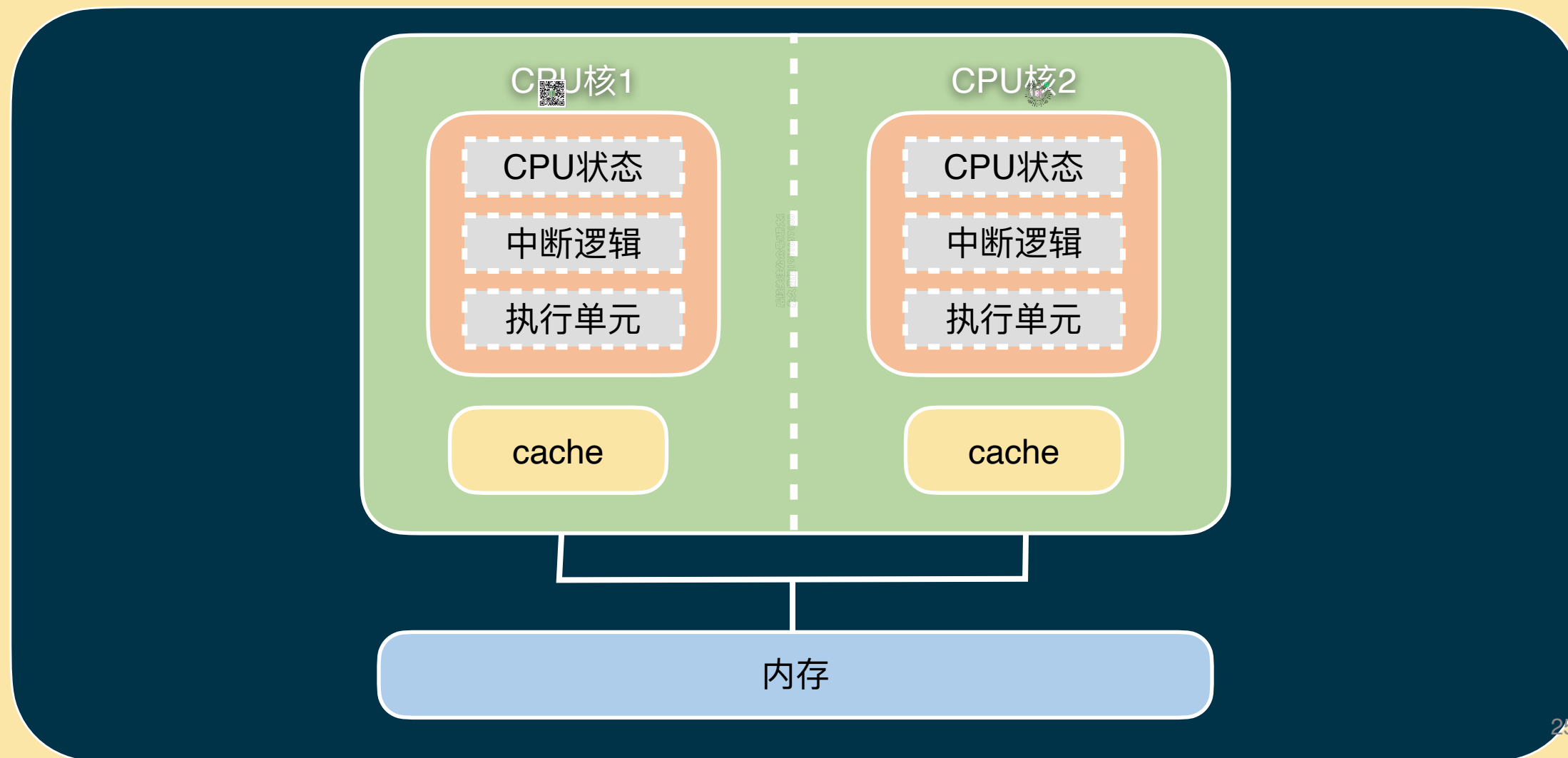
2.3 同时多线程

允许多个线程在同一时间内在单个处理器并行执行
通过在处理器内部共享和复用资源（如算术逻辑单元和寄存器文件）实现



3. 多核处理器

将多个处理单元集成到单个CPU中，每个处理单元称为一个核





4. 共享内存多处理器

(了解即可)

对称多处理器

两个或多个相同的处理器共享主内存、I/O设备等资源，并具有相同的访问内存的能力
处理器之间通过高速总线或其他数据连接方式相连，可以有效地协作处理任务
在SMP系统中，所有处理器共享一个操作系统，可以运行在任何处理器上，并共同访问所有资源

统一存储访问UMA多处理器

每个处理器对所有存储单元的访问时间大致相同
UMA的所有处理器共享同一内存空间
因此每个处理器中的cache都是共享内存
的副本，尤其需要注意各处理器中的cache一致性问题

非统一存储访问NUMA多处理器

主存被分割给不同的处理器，处理器访问自身附近的内存（本地内存）
比访问远程处理器的内存（远程内存）更快、更便宜